ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Үйлдлийн систем (F.CSM302)

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр:	UNIX Shell
Вариант:	№1
Хичээл заасан багш:	Г. Ганбат
Бие даалтын ажлыг гүйцэтгэсэн:	Оюутан: B231940067 (Д. Булгантуяа) Оюутан: B241940061 (Л. Ариунзаяа) Оюутан: B242270131 (У. Янжинлхам) Оюутан: B242270045 (Ч. Мөнхбаяр) Оюутан: B180930028 (Б. Энхбаяр) Оюутан: B241910015 (Т. Цэгц)

Улаанбаатар · 2026 он

Хураангуй

UNIX Shell Explorer нь C хэл дээр бичсэн командын интерпретатор бөгөөд хэрэглэгчийн оруулсан командыг child процесст гүйцэтгэнэ. Сурах бичгийн дагуу процесс үүсгэх, команд ажиллуулах, history, background гүйцэтгэл, оролт/гаралтын чиглүүлэлт болон pipe IPC-ийг хэрэгжүүлсэн. Түүнчлэн Python локал сервер, HTML интерфэйсээр команд өгөх, гаралтыг харах, процессын PID ба төлөвийн үйл явдлыг модоор дүрслэх боломжтой. Анхны хувилбарт MacBook дээр 8 автомат тест ажиллуулахад “PASS: 8 tests” гарсан. Дараа нь Python 3-ын subprocess модульд суурилсан автомат тестийн тоог 8-аас 16 болгон өргөтгөж, Ubuntu Linux (VirtualBox) болон macOS орчинд шалгахад 16 тест бүгд амжилттай гүйцэтгэгдэн “PASS: 16 tests” үр дүн гарсан. Энэ үр дүн нь тестээр хамрагдсан нөхцөлүүдийн зөв ажиллагааг харуулах бөгөөд олон pipe, shell expansion, бүрэн job control зэрэг боломжууд хэрэгжээгүй хэвээр байна.

1. Удиртгал ба зорилго

Энэ ажлын зорилго нь Operating System Concepts (10th edition) номын 3-р бүлгийн Programming Project 1 — UNIX Shell даалгаврыг гүйцэтгэж, процесс удирдах системийн дуудлагуудыг практикт туршиж ойлгох, хэрэглэгчийн оруулсан өөр өөр командыг ажиллуулах, процессын parent–child холбоог бодит PID-ээр харуулах явдал юм.

Ач холбогдол ба хамрах хүрээ. Shell нь хэрэглэгчийн команд болон үйлдлийн системийн процесс удирдлагын хоорондох үндсэн интерфэйс тул түүнийг хэрэгжүүлэх нь процесс үүсгэх, программ солих, процесс хоорондын харилцаа (IPC), файл тодорхойлогчийн удирдлагын ойлголтуудыг практикт холбож өгдөг. Энэхүү тайлан нь C хэл дээрх `osh` shell, Python локал сервер, Browser UI болон Process Tree дүрслэлийн зохиомж, хэрэгжилт, гар туршилт, 16 автомат тестийн үр дүн болон хэрэгжилтийн хязгаарлалтыг хамарна.

2. Сурах бичгийн шаардлага

Хүснэгт 2.1. Сурах бичгийн шаардлага ба хэрэгжүүлсэн арга

Шаардлага	**Хэрэгжүүлсэн арга**
Команд ба child процесс	`fork()`, `execvp()`, `waitpid()`
History	`!!` — өмнөх командыг давтах
Background	`&` — командын дуусахыг хүлээлгүй буцах
Оролт / гаралтын чиглүүлэлт	`open()` болон `dup2()`
Pipe IPC	`pipe()` ба `dup2()`, хоёр процессын хооронд
UI дүрслэл	PID, parent–child үйл явдлын бүртгэлээс process tree байгуулах

3. Онолын үндэс

Shell-ийн ажиллах үндсэн зарчим (REPL цикл): UNIX Shell нь хэрэглэгч болон үйлдлийн системийн цөм (Kernel) хоорондын гүүр болж ажилладаг командын интерпретатор юм. Shell нь үндсэн 3 үйлдлийг тасралтгүй гүйцэтгэдэг:

1. **Read (Унших):** Хэрэглэгчээс терминалаар дамжуулан командыг string хэлбэрээр уншиж авна.
2. **Eval (Үнэлэх/Боловсруулах):** Командыг задлан шинжилж (parsing), аргументуудыг ялган, холбогдох системийн дуудлагуудыг (fork, exec, pipe гэх мэт) дуудна.
3. **Print (Хэвлэх):** Тухайн командын гүйцэтгэлээс гарсан үр дүн эсвэл алдааны кодыг (exit status) дэлгэцэнд гаргана.

Хүснэгт 3.1. Shell-д хэрэглэсэн системийн дуудлагууд

Системийн дуудлага	Үүрэг
`fork()`	Эцэг процессоос хүүхэд процесс үүсгэнэ.
`execvp()`	Хүүхэд процессын программыг хэрэглэгчийн өгсөн командаар солино.
`waitpid()`	Эцэг процесс хүүхэд процесс дуустал хүлээнэ.
`pipe()`	Хоёр процессын хооронд нэг чиглэлтэй byte stream үүсгэнэ.
`dup2()`	Стандарт оролт, гаралтыг файл эсвэл pipe руу дахин чиглүүлнэ.

4. Системийн зохиомж

Системийн архитектур: Системийн архитектур нь Browser UI (эсвэл GitHub Pages дээр байрлах интерактив симулятор) → Python локал сервер → C хэл дээр бичсэн osh команд гүйцэтгэгч гэсэн дарааллаар ажилладаг.

- **Интерактив симулятор ба GitHub Pages:** вэб суурьтай интерактив симулятор нь хэрэглэгчдэд сервер асаах шаардлагагүйгээр шууд хөтөч дээрээс UNIX Shell-ийн команд болон процесс модыг турших боломжийг олгож байна. Уг симуляторыг GitHub Pages руу байрлуулсан бөгөөд дараах линкеэр орж ашиглах боломжтой

GitHub Pages холбоос: https://bulgaa777.github.io/II-4-1-UNIX-Shell/Interactive/osh_HowItWorks.html
GitHub repository: https://github.com/bulgaa777/II-4-1-UNIX-Shell

Архитектурын онцлог ба хязгаарлалт: Энэхүү систем нь вэб суурьтай интерфэйсээр (Browser UI) дамжуулан өгөгдөл хүлээн авч, Python локал серверээр дамжуулан C хэл дээр бичсэн osh програмыг ажиллуулдаг. Команд бүрт шинээр osh процесс үүсэж ажилладаг тул энэ нь уламжлалт интерактив shell шиг нэг сесс дээр орчны хувьсагч (environment variables) эсвэл ажиллаж буй директорийн төлөвийг (cd командын үр дүнг) дараагийн команд руу шууд хадгалж үлдэхгүй сул талтай. Үүнийг шийдвэрлэхийн тулд процесст хандах үйл явдлыг файл руу бичиж, UI талд Process Tree хэлбэрээр бодит цаг хугацаанд дүрсэлдэг.

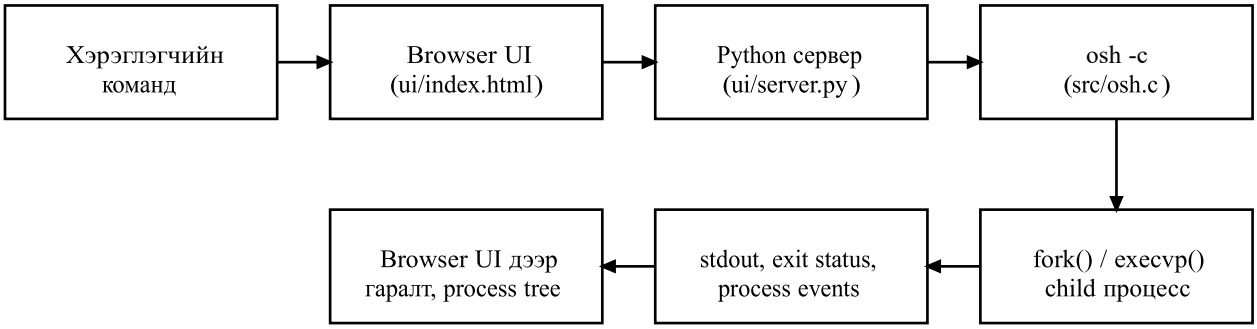
Хүснэгт 4.1. Системийн бүрэлдэхүүн

Бүрэлдэхүүн	Файл	Үүрэг
Frontend	`ui/index.html`	Команд оруулах талбар, гаралт, process tree, history харуулна.

Backend	`ui/server.py`	Локал HTTP сервер; C программыг ажиллуулж гаралт, event бүртгэлийг дамжуулна.
Shell	`src/osh.c`	Команд задлах, fork/exec, pipe, redirection хэрэгжүүлнэ.
Тест	`tests/test_shell.py`	16 автомат тест (8 үндсэн, 8 нэмэлт).

Үйл явдлын файлд цаг, PID, PPID, state, command хадгалагдах бөгөөд UI үүнийг process tree болгон дүрслэнэ.

Өгөгдлийн урсгал. Хэрэглэгчийн команд → Browser UI → Python сервер → `osh -c` → fork/exec → stdout, exit status, process events → Browser UI.



Зураг 4.1. Системийн өгөгдлийн урсгал

Одоогийн хэрэгжилтэд команд бүрт шинэ `osh` процесс үүсдэг тул энэ нь нэг тасралтгүй shell session биш болно.

5. Хэрэгжилт ба орчин

Хэрэгжилтэд C11 (Clang/cc), Python 3, HTML/CSS/JavaScript ашигласан. Программыг macOS дээр VS Code болон Chrome ашиглан гараар ажиллуулж шалгасан.

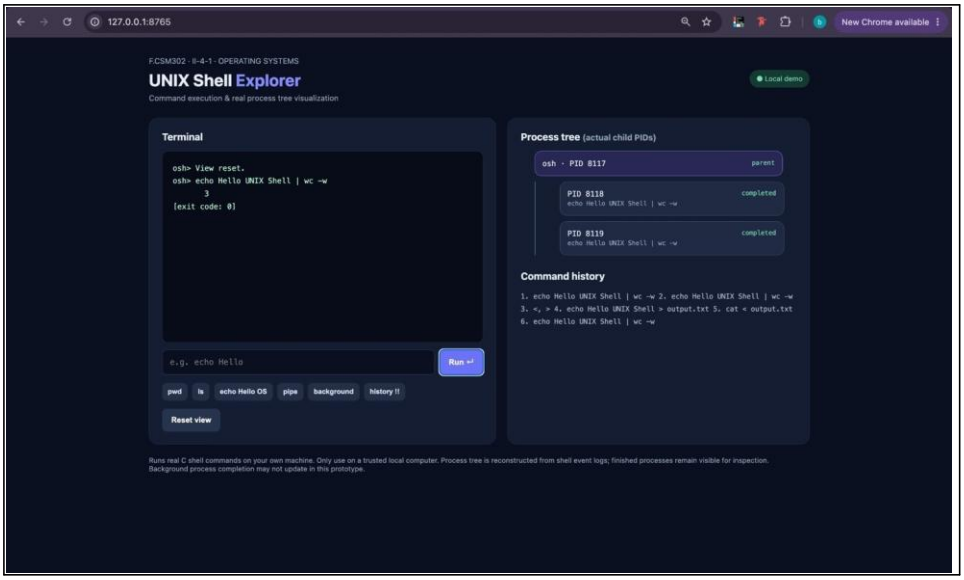
Makefile-ийн compile команд: `cc -std=c11 -Wall -Wextra -O2 -o osh src/osh.c`

Веб UI-г `python3 ui/server.py` командаар асааж, `http://127.0.0.1:8765` хаягаар нээнэ.

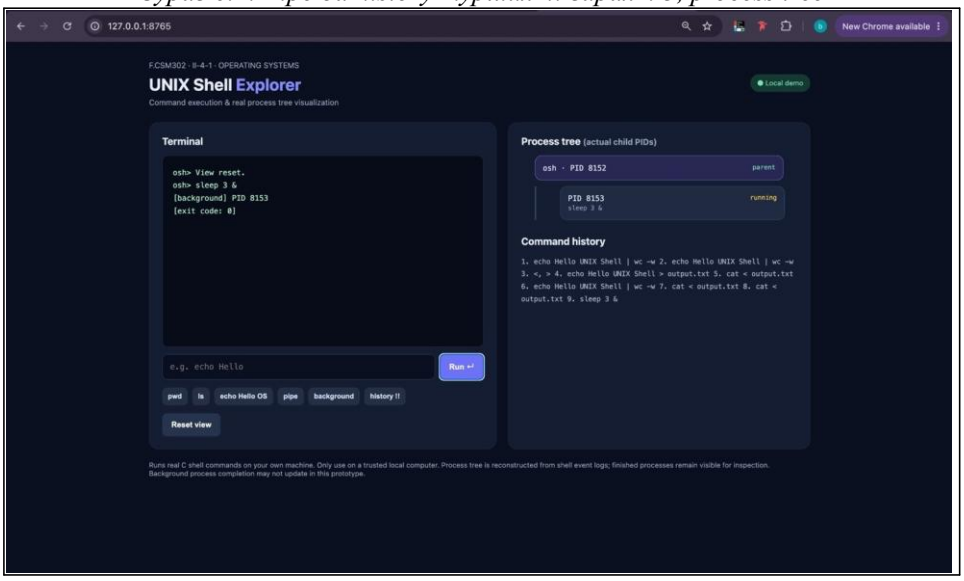
6. Туршилт ба UI дүрслэл

UNIX Shell Explorer-ийн Browser UI нь **Terminal**, **Process tree**, **Command history** гэсэн үндсэн хэсгээс бүрдэнэ. Хэрэглэгч команд оруулах талбарт дурын команд бичих, эсвэл `pwd`, `ls`, `echo Hello OS`, `pipe`, `background`, `history !!` гэсэн бэлэн товчийг ашиглан командыг өөрчилж туршина. **Reset view** товч нь дүрслэлийг цэвэрлэж, шинээр турших боломж олгоно.

Команд бүрийн дараа дараах зүйлсийг ажиглана: Terminal хэсэгт командын гаралт ба exit code; Process tree хэсэгт `osh` parent процесс болон түүний child процессуудын бодит PID ба төлөв; Command history хэсэгт өмнө оруулсан командуудын жагсаалт. Pipe-тай командын үед хоёр child процесс нэг parent процессын дор харагдана (Зураг 6.1). Background командын үед child процесс running төлөвтэй үүсэж, shell хүлээлгүйгээр буцна (Зураг 6.2).



Зураг 6.1. Pipe ба history туршилт: гаралт 3, process tree



Зураг 6.2. Background: sleep 3 &, PID 8153 үүссэн, төлөв running

7. Туршилтын арга

Туршилтыг хоёр хэлбэрээр гүйцэтгэсэн: (1) Browser UI болон терминалаар командыг гараар оруулж, гаралт, exit code, Process Tree-ийн төлөвийг ажиглах гар туршилт; (2) `tests/test_shell.py` файлд тодорхойлсон 16 автомат тест.

7.1. Туршилтын орчин

Хүснэгт 7.1. Туршилтын орчин

Орчин	**Гүйцэтгэсэн**	**Хэрэгсэл**	**Туршилтын агуулга**
macOS (MacBook Air)	Д. Булгантуяа	VS Code Terminal, Chrome, Python 3, cc	Гар туршилт (Хүснэгт 8.1), анхны хувилбарын 8 автомат тест
Ubuntu Linux (VirtualBox)	Б. Энхбаяр	Terminal, Python 3	16 автомат тест, тест тус бүрийн төлөвтэй гаралт

Орчин	Гүйцэтгэсэн	Хэрэгсэл	Туршилтын агуулга
macOS (MacBook Air)	Д. Булгантуяа	VS Code Terminal, make, Python 3	main branch-ийг шинэчилсний дараах 16 автомат тест

7.2. Гар туршилтын өгөгдөл

Гар туршилтад pipe, history, output/input redirection, background гүйцэтгэл болон `make test`-ийг хамарсан командуудыг өгч, Хүснэгт 8.1-д заасан хүлээгдсэн үр дүнтэй харьцуулсан. Ажиглалтыг UI-ийн бодит screenshot-оор баримтжуулсан (Зураг 6.1, 6.2, Хавсралт А, В).

7.3. Автомат тестийн арга

Автомат тестийг Python 3-ын `subprocess` модуль ашиглан `tests/test_shell.py` файлд хэрэгжүүлсэн. Тест бүр compile хийсэн `osh` программыг `-c` горимд нэг командтайгаар, эсвэл стандарт оролтоор олон мөр дамжуулж интерактив горимд ажиллуулаад буцаах код (returncode), стандарт гаралт (stdout), алдааны гаралт (stderr)-ыг `check()` функцээр шалгана. Redirection-ий тестүүд түр хавтаст (`tempfile.TemporaryDirectory`) ажилладаг тул төслийн файлд нөлөөлөхгүй бөгөөд процесс бүрд 8 секундын хугацааны хязгаар тавьсан.

Нөхцөл биелээгүй тохиолдолд `AssertionError` үүсч, тухайн тестийн нэрээр зогсоно. Бүх шалгалт амжилттай болсон үед л эцсийн нэгдсэн мөр хэвлэгдэнэ. Тестийг төслийн үндсэн хавтсанд дараах командаар ажиллуулна; Makefile-ийн `test` зорилт эхлээд `osh`-ийг compile хийж, дараа нь тестийн файлыг ажиллуулна:

```
$ make test
python3 tests/test_shell.py
```

Анхны хувилбарт 8 үндсэн тест (1–8) байсан бөгөөд Б. Энхбаяр алдаатай болон захын оролтыг шалгах нэмэлт 8 тест (9–16) нэмж, нийт 16 болгосон. Тест тус бүрийн зорилго, хүлээгдсэн үр дүнг Хүснэгт 8.2-т үзүүлэв.

Ubuntu орчинд тест тус бүрийн төлөвийг харахын тулд `check()` функцэд “PASS - <тестийн нэр>” мөр хэвлэх үйлдлийг `sed` командаар нэмсэн түр хуулбар (`tests/_verbose.py`) үүсгэж ажиллуулаад устгасан (Зураг 8.1). Уг өөрчлөлт нь тестийн нөхцөлийг өөрчлөөгүй, зөвхөн гаралтыг дэлгэрэнгүй болгосон.

8. Үр дүн

8.1. Гар туршилтын үр дүн

Гар туршилтын ажиглалтыг Хүснэгт 8.1-д нэгтгэв. Ажиглалт нь Зураг 6.1, 6.2, Хавсралт А, В-ийн бодит screenshot болон терминалын үр дүнд үндэслэсэн.

Хүснэгт 8.1. Гар туршилтын үр дүн

Тест	Хүлээгдсэн үр дүн	Ажиглагдсан үр дүн
`echo Hello UNIX Shell | wc -w`	3	3, exit code 0

`!!`	Өмнөх команд дахин ажиллана	Өмнөх команд дахин ажиллаж, history-д давтагдсан бичлэг нэмэгдсэн
`echo Hello UNIX Shell > output.txt`	output.txt файл үүснэ	exit code 0; гаралт файлд чиглэгдсэн
`cat < output.txt`	Hello UNIX Shell	Hello UNIX Shell гарсан (Хавсралт А)
`sleep 3 &`	PID үүсч, шууд буцна	Child PID 8153, төлөв running
`make test`	Автомат тестүүд тэнцэнэ	PASS: 8 tests (анхны хувилбар, Зураг 8.3); PASS: 16 tests (эцсийн хувилбар, Зураг 8.1, 8.2)

8.2. UNIX Shell-ийн 16 автомат тестийн үр дүн

Нэмэлт тестүүдийг нэгтгэсний дараа 16 автомат тестийг Ubuntu Linux болон macOS гэсэн хоёр өөр орчинд ажиллуулсан. Хоёр орчинд 16 тест бүгд амжилттай гүйцэтгэгдсэн (Хүснэгт 8.2, 8.3).

Хүснэгт 8.2. Автомат тестүүд, шалгах зорилго ба үр дүн

№	**Тест**	**Шалгах зорилго ба оролт**	**Хүлээгдсэн үр дүн**	**Ажиглагдсан төлөв (Ubuntu / macOS)**
Үндсэн функцийн тестүүд (анхны хувилбар)				
1	`echo`	Энгийн командыг child процесст гүйцэтгэх: `echo hello`	exit code 0, stdout = `hello`	PASS / PASS
2	`pipe`	Хоёр командыг pipe-аар холбох: `echo hello | wc -c`	exit code 0, stdout = `6`	PASS / PASS
3	`output redirect`	Гаралтыг файл руу чиглүүлэх: `echo sample > output.txt` (түр хавтаст)	exit code 0	PASS / PASS
4	`input redirect`	Файлаас оролт авах: `cat < output.txt`	stdout = `sample`	PASS / PASS
5	`empty history`	Түүх хоосон үед `!!` гүйцэтгэх	stderr-д “No commands in history” мэдээлэл	PASS / PASS
6	`invalid pipe`	Бүрэн талын командгүй pipe: `echo hi |`	exit code ≠ 0	PASS / PASS
7	`missing file`	Байхгүй файлаас оролт авах: `cat < /this_file_does_not_exist_123`	exit code ≠ 0	PASS / PASS
8	`history replay`	Интерактив горимд `echo first`, `!!`, `exit` дараалан оруулах	stdout-д “first” дор хаяж 2 удаа	PASS / PASS

№	Тест	Шалгах зорилго ба оролт	Хүлээгдсэн үр дүн	Ажиглагдсан төлөв (Ubuntu / macOS)
Нэмэлт болон захын нөхцөлийн тестүүд (Б. Энхбаяр)				
9	`invalid command`	Байхгүй команд: `no_such_cmd_xyz_123`	exit code ≠ 0, stderr хоосон биш	PASS / PASS
10	`empty input`	Хоосон команд (`""`)	exit code 0, stdout хоосон	PASS / PASS
11	`whitespace input`	Зөвхөн хоосон зайтай оролт (`"  "`)	exit code 0	PASS / PASS
12	`missing file message`	Байхгүй файлын үед хэрэглэгчид мэдээлэл өгөх эсэх	stderr хоосон биш	PASS / PASS
13	`redirect without filename`	Файлын нэргүй output redirection: `ls >`	exit code ≠ 0	PASS / PASS
14	`pipe without left command`	Зүүн талын командгүй pipe: `| wc -c`	exit code ≠ 0	PASS / PASS
15	`redirect to bad path`	Байхгүй хавтас руу гаралт: `echo x > /no_such_dir_123/out.txt`	exit code ≠ 0	PASS / PASS
16	`blank lines then exit`	Интерактив горимд хоосон мөрүүдийн дараа `exit`	exit code 0	PASS / PASS

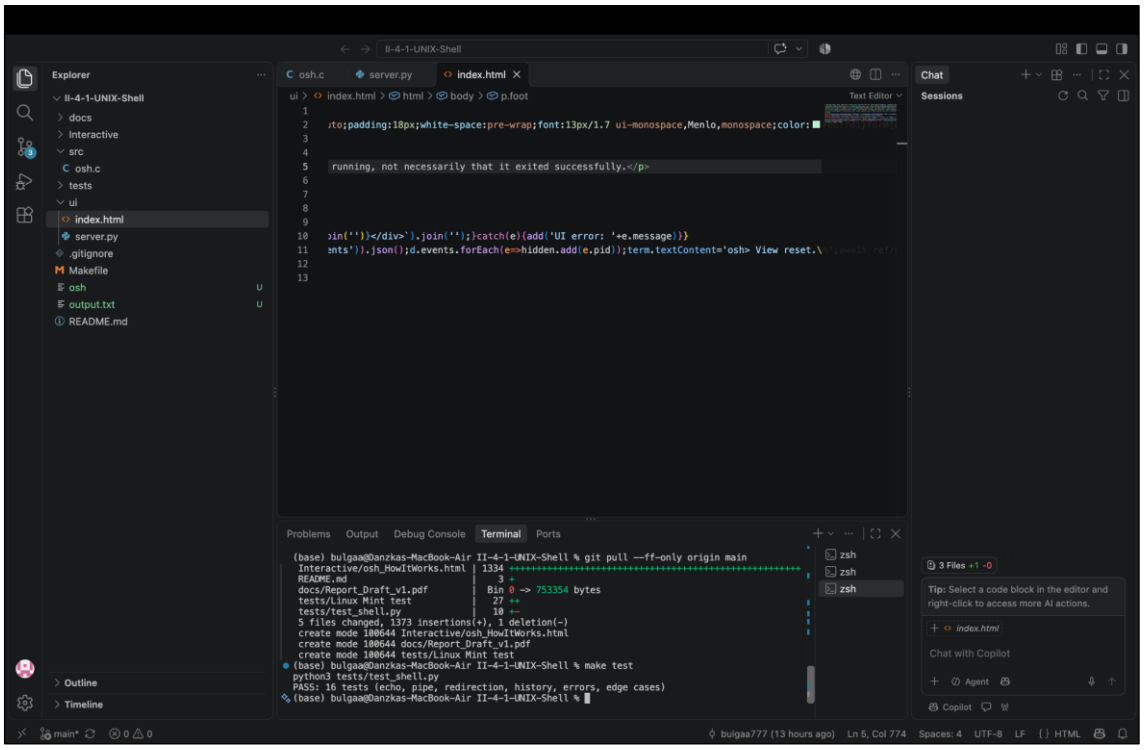
Тайлбар: Хүлээгдсэн үр дүнг `tests/test_shell.py`-ийн шалгах нөхцөлөөс авсан. Ubuntu дээрх тест тус бүрийн PASS төлөв Зураг 8.1-д харагдана. macOS дээрх “PASS: 16 tests” мөр нь бүх шалгалт давсан үед л хэвлэгддэг тул 16 тест тус бүр амжилттай болсныг илэрхийлнэ (Зураг 8.2).

Хүснэгт 8.3. Автомат тестийн нэгдсэн үзүүлэлт

Үзүүлэлт	Ubuntu Linux (VirtualBox)	macOS
Үндсэн автомат тест	8	8
Нэмэлт автомат тест	8	8
Нийт тест	16	16
Амжилттай тест	16	16
Амжилтгүй тест	0	0
Амжилтын хувь	100%	100%
Гүйцэтгэсэн	Б. Энхбаяр	Д. Булгантуяа
Ажиллуулсан команд	`python3 tests/_verbose.py` (түр хуулбар)	`make test`

```
enkhbayar@enkhbayar-VirtualBox:~/os-lab/II-4-1-UNIX-Shell$ sed 's/^def check(cond, msg):/def check(cond, msg):\n print
("PASS" if cond else "FAIL", "-", msg)/' tests/test_shell.py > tests/_verbose.py
python3 tests/_verbose.py
rm tests/_verbose.py
PASS - echo
PASS - pipe
PASS - output redirect
PASS - input redirect
PASS - empty history
PASS - invalid pipe
PASS - missing file
PASS - history replay
PASS - invalid command
PASS - empty input
PASS - whitespace input
PASS - missing file message
PASS - redirect without filename
PASS - pipe without left command
PASS - redirect to bad path
PASS - blank lines then exit
PASS: 16 tests (echo, pipe, redirection, history, errors, edge cases)
enkhbayar@enkhbayar-VirtualBox:~/os-lab/II-4-1-UNIX-Shell$
```

Зураг 8.1. Ubuntu Linux (VirtualBox) орчинд 16 автомат тест тус бүрийн PASS төлөв ба нэгдсэн үр дүн (гүйцэтгэсэн: Б. Энхбаяр)



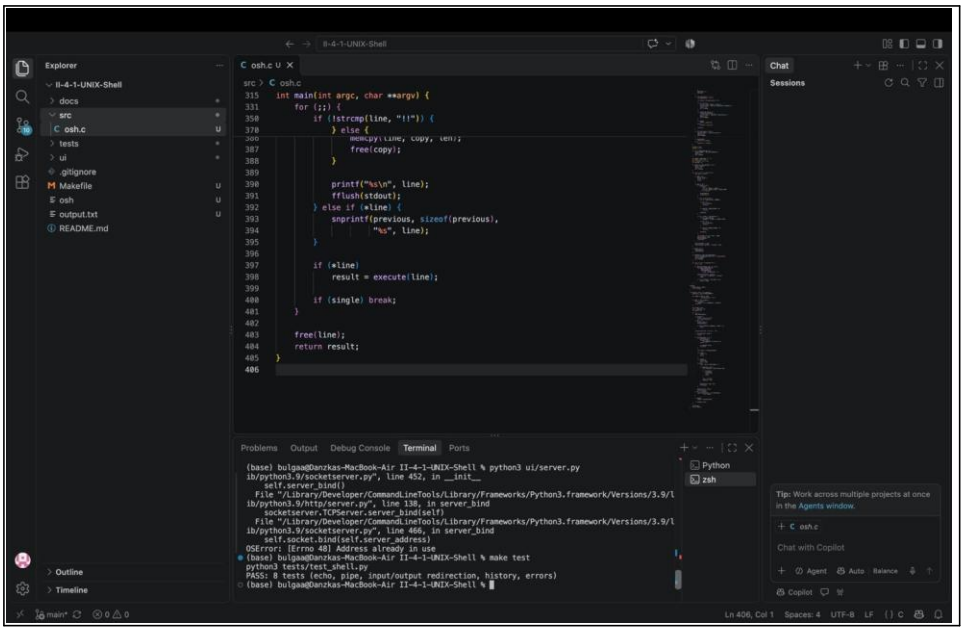
Зураг 8.2. macOS орчинд `git pull --ff-only origin main` хийсний дараа `make test` командаар 16 автомат тест амжилттай гүйцэтгэгдсэн байдал (гүйцэтгэсэн: Д. Булгантуяа)

Зураг 8.2-т Д. Булгантуяа GitHub-ийн main branch-ийн шинэчлэлтийг татсаны дараа `make test` ажиллуулсан байдал харагдана; татсан өөрчлөлтөд `tests/test_shell.py` файл орсон бөгөөд терминалд “PASS: 16 tests (echo, pipe, redirection, history, errors, edge cases)” гэсэн үр дүн гарсан.

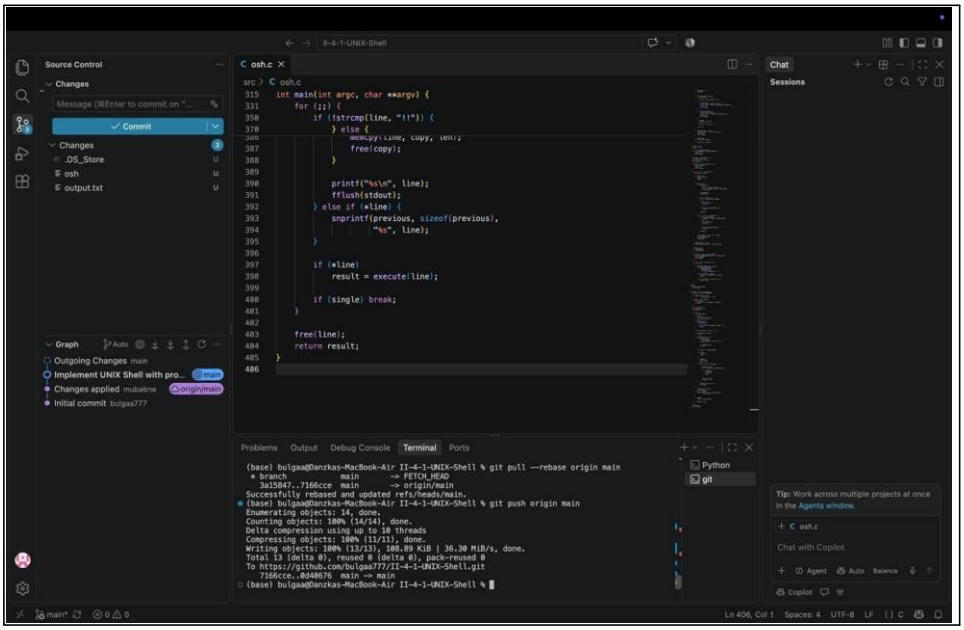
8.3. Анхны хувилбарын үр дүн ба GitHub нотолгоо

`make test` командаар ажиллах анхны хувилбарын 8 автомат тест нь echo, pipe, оролт/гаралтын чиглүүлэлт, history болон алдааны тохиолдлыг шалгадаг байсан бөгөөд MacBook дээр “PASS: 8 tests” үр дүн гарсан (Зураг 8.3). Энэ зураг нь өмнөх хувилбарын түүхэн нотолгоо бөгөөд эцсийн баталгаажуулалтад Зураг 8.1, 8.2-ыг ашиглана. Эх код, тест, баримт бичгийг GitHub repository-д

байршуулсан (Зураг 8.4). Нэмэлт 8 тестийг Б. Энхбаяр `79b01f1` commit-оор `test/edge-cases` branch-д нэмж, GitHub Pull Request #1-ээр main branch-д нэгтгэсэн (merge commit `e8df8b3`).



Зураг 8.3. Анхны хувилбарын 8 тестийн үр дүн



Зураг 8.4. git pull --rebase болон git push амжилттай

9. Хэлэлцүүлэг ба хязгаарлалт

9.1. Туршилтын үр дүнгийн тайлбар

16 автомат тест Ubuntu Linux болон macOS дээр бүгд амжилттай гүйцэтгэгдсэн нь `osh`-ийн үндсэн функцүүд—энгийн команд гүйцэтгэх, хоёр командын pipe, оролт/гаралтын чиглүүлэлт, `!!` history—тестэд тодорхойлсон оролт дээр хүлээгдсэн гаралт ба буцаах кодыг өгч байгааг харуулж байна. Түүнчлэн нэг C эх код Linux болон macOS орчинд compile хийгдэж, тестээр хамрагдсан нөхцөлд

ижил үр дүн өгсөн нь программын зөөвөрлөх чадварыг (portability) тодорхой хэмжээнд баталж байна.

Анхны 8 тестийн ихэнх нь хэвийн ажиллагааг шалгадаг байсан бол нэмэлт 8 тест нь алдаатай болон захын оролтод төвлөрсөн: байхгүй команд, хоосон болон зөвхөн хоосон зайтай оролт, файлын нэргүй redirection, зүүн талын командгүй pipe, байхгүй хавтас руу чиглүүлэх, хоосон мөрийн дараах `exit`. Эдгээр нь shell алдаатай оролтыг танин буцаах кодоор мэдээлж, хоосон оролтыг алдаа гэж үзэхгүй байгаа эсэхийг шалгадаг. Ялангуяа `invalid command` болон `missing file message` тестүүд нь зөвхөн буцаах код бус stderr-д мэдээлэл гарч буй эсэхийг шалгаснаар багшийн зааврын “буруу болон захын оролтыг шалгаж, ойлгомжтой мэдээлэх” шаардлагатай уялдана.

Ubuntu дээрх туршилт нь тест тус бүрийн төлөвийг ил харуулсан бол macOS дээрх туршилт нь Pull Request #1-ээр нэгтгэсэн main branch-ийг өөр компьютерт татаж, `make test`-ээр давтан шалгасан нотолгоо болно. Ингэснээр нэмэлт тестүүдийг оруулсны дараа анхны 8 шалгалт хэвийн ажилласаар байгаа (regression үүсээгүй) нь хоёр орчинд баталгаажсан.

Гэхдээ 16/16 үр дүн нь программ бүх нөхцөлд алдаагүй ажиллана гэсэн баталгаа биш. Тестүүд тогтмол, цөөн тооны оролт ашигладаг бөгөөд зарим тест (`invalid pipe`, `redirect without filename`, `redirect to bad path` г.м.) зөвхөн буцаах код тэгээс ялгаатай эсэхийг шалгаж, алдааны мэдээллийн агуулгыг шалгадаггүй. Background (`&`) гүйцэтгэл, Python сервер, Browser UI болон Process Tree-ийн дүрслэл автомат тестэд хамрагдаагүй тул эдгээрийг гар туршилтаар (Хүснэгт 8.1, Хавсралт B) шалгасан.

9.2. Хязгаарлалт

- Команд бүрт шинэ `osh` (parent) процесс үүсдэг тул UI нь нэг тасралтгүй shell session биш.
- Process tree нь процессын event log-д суурилсан дүрслэл юм. osh нь child процесс үүсгэх бүрд цаг, PID, PPID, төлөв, командыг бүртгэлд бичиж, UI энэ бүртгэлээс модыг сэргээн байгуулдаг. Иймд дүрслэл нь kernel-ийн процессын хүснэгтийг шууд уншсан бус, бүртгэгдсэн үйл явдлын түүх юм.
- `running` төлөв нь child процесс үүсэх үед бүртгэгддэг бол `finished` төлөв нь сервер тухайн PID цаашид ажиллахгүй байгааг шалгасны дараа тогтоогдоно. Иймд хоёр төлөвийн хооронд дараагийн UI refresh хүртэл хугацааны зөрүү гарч болно.
- Шинэчилсэн Python сервер background процессын PID-ийг шалгаж, running төлөвийг finished болгон шинэчилдэг болсон. Гэхдээ finished төлөв нь амжилттай exit code 0 гэдгийг батлахгүй.
- Parser нь сургалтын хялбар хувилбар тул олон pipe, shell expansion, бүрэн job control-ийг дэмжихгүй.
- History нь зөвхөн сүүлийн командыг хадгалж, `!!`-ээр давтах боломжтой; дугаараар хандах бүрэн түүх хэрэгжээгүй.
- 16/16 автомат тест нь зөвхөн тестээр хамрагдсан нөхцөлүүдийн зөв ажиллагааг харуулах бөгөөд бүх боломжит оролтод алдаагүй ажиллах баталгаа биш.

10. Дүгнэлт

Сурах бичгийн үндсэн системийн дуудлага болон shell-ийн гол үйлдлүүдийг хэрэгжүүлж, бодит PID бүхий процессын дүрслэлтэй холбосон. Pipe, history, redirection болон background процессын үүсэлтийг туршсан. Нэмэлт засварын дараа background процесс дуусахад finished төлөв гардаг болсон (Хавсралт B).

`osh` нь `fork()`-оор child процесс үүсгэж, `execvp()`-ээр командын программыг ачаалж, `waitpid()`-ээр foreground процессыг хүлээдэг бол `&` бүхий команд хүлээлгүйгээр prompt руу буцна. Оролт/гаралтын чиглүүлэлтийг `open()` ба `dup2()`, хоёр командын холбоог `pipe()` ба `dup2()`-оор хэрэгжүүлж, `!!` командаар сүүлийн командыг давтдаг. Python локал сервер нь хэрэглэгчийн командыг `osh`-д дамжуулж, event log-ооc Browser UI-д Process Tree байгуулна.

Автомат тестийн тоог 8-аас 16 болгон өргөтгөснөөр хэвийн ажиллагаанаас гадна алдаатай болон захын оролтын боловсруулалт шалгагдах болсон. Ubuntu Linux болон macOS орчинд 16/16 тест амжилттай гүйцэтгэгдсэн нь тестээр хамрагдсан нөхцөлд shell зөв, давтагдахуйц ажиллаж байгааг нотолсон боловч бүх оролтын баталгаа биш юм.

Цаашид нэг тасралтгүй shell session, олон pipe, shell expansion, бүрэн job control, background процессын exit status-ийг UI-д харуулах, мөн UI ба background гүйцэтгэлийг хамарсан автомат тест нэмснээр системийн найдвартай байдлыг сайжруулах боломжтой.

Ашигласан материал

1. F.CSM302 Үйлдлийн систем, Бие даалт 1 удирдамж, 2026.

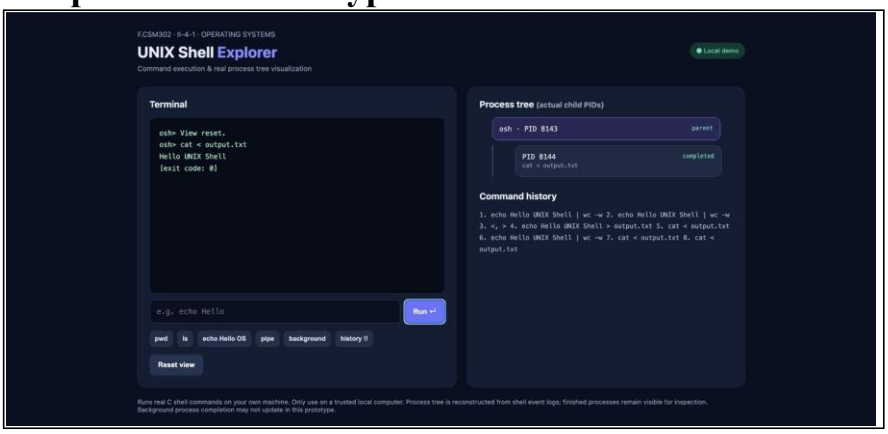
2. F.CSM302, Сэдэв ба шүүмжийн хуваарилалт, II-4-1 UNIX Shell.

3. A. Silberschatz, P. B. Galvin, G. Gagne. Operating System Concepts, 10th ed. Wiley, 2018 (Chapter 3, Programming Project 1 — UNIX Shell).

4. Python Software Foundation. subprocess — Subprocess management. Python 3 documentation.

5. II-4-1-UNIX-Shell GitHub repository: source, tests, docs. https://github.com/bulgaa777/II-4-1-UNIX-Shell

Хавсралт A. Input Redirection туршилт



Зураг A. cat < output.txt команд output.txt файлаас уншиж, гаралтыг харуулсан

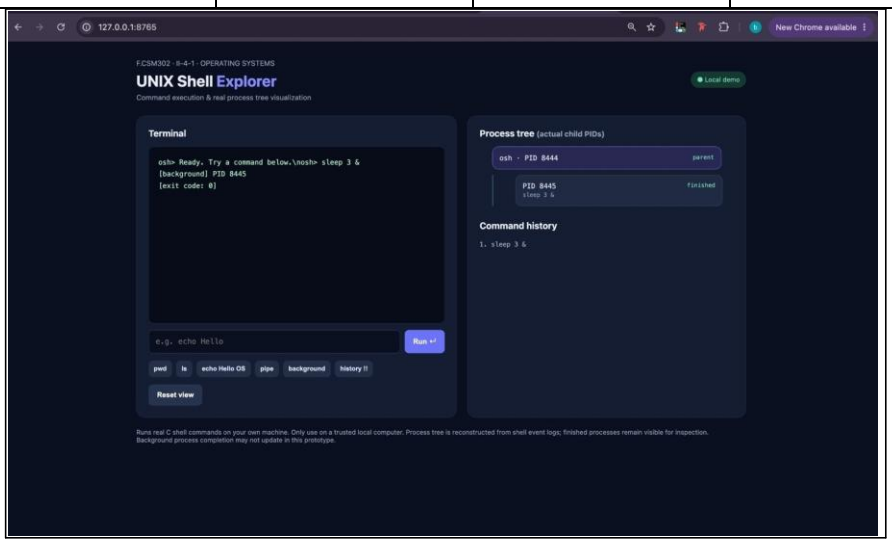
Ажигласан гаралт: Hello UNIX Shell, exit code 0.

Хавсралт B. Background төлөвийн шинэчлэл

Өмнөх зурагт (Зураг 6.2) child процесс running гэж харагдсан. Шинэчилсэн `ui/server.py` нь процесс дууссаныг шалгаж, дараагийн UI refresh-д finished төлөвөөр дүрсэлдэг. Үүнийг доорх бодит screenshot харуулна. Энэ screenshot нь туршилтыг дахин ажиллуулсан үеийнх тул PID нь Зураг 6.2-оос өөр байна.

Хүснэгт B.1. Background туршилтын PID

Туршилт	Parent PID	Child PID	Төлөв
`sleep 3 &`	8444	8445	finished



Зураг B. sleep 3 &: процесс дууссаны дараа Process Tree-д finished төлөв

Ажиглалт: Процесс дууссаны дараа Process Tree-д finished гэж харагдсан. **Тайлбар:** finished төлөв нь процесс цаашид ажиллахгүй байгааг илэрхийлнэ. Амжилттай дууссан эсэхийг exit status-аар тусад нь шалгана.

Хавсралт С. Багийн гишүүдийн оролцоо

Хүснэгт С.1. Багийн гишүүдийн оролцоо

Гишүүн	**Хариуцсан ажил**	**GitHub нотолгоо (commit / файл)**
Д. Булгантуяа (B231940067)	UNIX Shell-ийн үндсэн функцүүд, Python сервер, Web UI болон Process Tree-ийн хэрэгжүүлэлт, сайжруулалт дээр ажилласан. Мөн macOS орчинд тестүүдийг баталгаажуулж, багийн зохион байгуулалт, GitHub Pages болон төслийн нэгтгэлийг хариуцсан.	src/osh.c, ui/, tests/; 83a2fd2, PR #1 merge
Л. Ариунзаяа (B241940061)	Тайлангийн эхний хувилбарыг бэлтгэж, UNIX Shell-ийн онолын үндэс, системийн дуудлагууд, архитектур болон өгөгдлийн урсгалын тайлбарыг боловсруулсан.	docs/Report_Draft_v1.pdf; 70e47d0
У. Янжинлхам (B242270131)	Туршилтын аргачлал, 16 автомат тестийн үр дүн, Ubuntu/macOS туршилтын хүснэгт, дүгнэлт болон багийн оролцоог боловсруулсан. Тайлангийн агуулга, форматыг нэгтгэн эцэслэсэн.	docs/II-4-1_Unix_Shell_Report_Final.pdf 622a5bf
Ч. Мөнхбаяр (B242270045)	UNIX Shell-ийн ажиллагааг тайлбарлах интерактив симулятор, процессын архитектурын дүрслэлийг хөгжүүлсэн. README-ийн Quick Start, ажиллуулах заавар болон GitHub Pages холбоосыг шинэчилсэн.	Interactive/osh_HowItWorks.html, README.md; 6ec50f1, 9a94388
Б. Энхбаяр (B180930028)	Алдаатай болон захын оролтыг шалгах 8 нэмэлт автомат тест бичиж, нийт 16 тестийг Ubuntu VirtualBox орчинд ажиллуулан баталгаажуулсан. Тестийн үр дүн, гарын авлага бэлтгэсэн.	tests/test_shell.py; 79b01f1, PR #1
Т. Цэгц (B241910015)	UNIX Shell Explorer-ийн хэрэглэгчийн гарын авлагыг боловсруулж, программ суулгах, Shell болон Web UI ажиллуулах, команд болон автомат тест ашиглах алхамчилсан зааврыг бэлтгэсэн. Linux Mint орчинд туршилт хийсэн.	docs/II-4-1_Manual.pdf; commit ID — тодруулах шаардлагатай

Тэмдэглэл: Commit ID-г GitHub repository-ийн commit түүхээс (2026-10-28-ны байдлаар) шалгав.